

Homework: attacking a CNN digit model

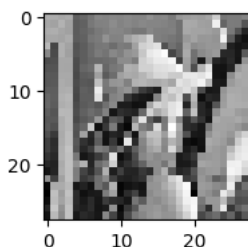
In this homework, you will train a CNN model for digit classification, then freeze the model. The goal is then to take an image (e.g., a human face), and modify it's pixels to force the model to classify it as a digit!

1. Import the 512 x 512 Lenna image (csv file in canvas)
2. Downsize it to 28 x 28
3. Train a digit model/or import one that classify a 28x28 image as 0 to 9. Freeze the weights. For this step, you may use the SimpleCNN in our lecturenote
4. What's the predicted label of Lenna? What's the probability of that label (using softmax), is it high (e.g., > 0.9)?
5. Now modify the Lenna image to Lenna2 so as to maximize the probability of it being classified as 8.
6. What's the predicted label of Lenna2? What's the probability of that label (using softmax)
7. What's the difference between the original image and the new image in terms of mean square error (MSE)?
8. Now create Lenna3, which no only maximizes the probability of it being classified as 8, but also minize the difference between it and Lenna. You can define the loss function as a weighted sum of cross entropy loss and the MSE loss
9. What's the predicted label of Lenna3? What's the probability of that label (using softmax)
10. What's the difference between the Lenna and Lenna3 in terms of mean square error (MSE)? How does it compare with MSE between lenna and lenna2
11. Plot Lenna, Lenna2, Lenna3 side-by-side, and visually compare these 3 images and give your observation of whether your attach is successful (able to for any label yet the image still looks similar to original)

```
# 1. import Lenna and 2. shrink it down to 28x28
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
import pandas as pd
import matplotlib.pyplot as plt
from torchvision import datasets, transforms

lenna = pd.read_csv("/content/lenna.csv", header = None).values
lenna = F.interpolate(torch.tensor(lenna, dtype=torch.float32).unsqueeze(0).unsqueeze(0), size=(28,28))
plt.figure(figsize=(2, 2))
plt.imshow(lenna[0][0], cmap = 'gray')
```

<matplotlib.image.AxesImage at 0x7ba0d3009730>



```
# 3. Train a digit model/or import one that classify a 28x28 image as 0 to 9.
# Freeze the weights. For this step, you may use the SimpleCNN in our lecture note. Also see lecture
# note on how to freeze weights (set params.requires_grad = False)
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F
from torchvision import datasets, transforms

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 32, 3, 1) # (28-2)=26
        self.conv2 = nn.Conv2d(32, 64, 3, 1) # (26-2)=24
        self.pool = nn.MaxPool2d(2) # 24 -> 12
        self.fc1 = nn.Linear(64 * 12 * 12, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(F.relu(self.conv2(x)))
        x = torch.flatten(x, 1)
        x = F.relu(self.fc1(x))
        x = self.fc2(x)
        return x
```

```
def train(model, device, train_loader, optimizer, epoch):
    for batch_idx, (data, target) in enumerate(train_loader):
        data, target = data.to(device), target.to(device)
        optimizer.zero_grad()
        output = model(data)
        loss = F.cross_entropy(output, target)
        loss.backward()
        optimizer.step()
        if batch_idx % 100 == 0:
            print(f'Train Epoch: {epoch} [{batch_idx * len(data)} / {len(train_loader.dataset)} '
                  f'({100. * batch_idx / len(train_loader):.0f}%) \tLoss: {loss.item():.6f}')
```

```
def test(model, device, test_loader):
    model.eval()
    test_loss = 0
    correct = 0
    with torch.no_grad():
        for data, target in test_loader:
            data, target = data.to(device), target.to(device)
            output = model(data)
            test_loss += F.cross_entropy(output, target, reduction='sum').item()
            pred = output.argmax(dim=1, keepdim=True)
            correct += pred.eq(target.view_as(pred)).sum().item()

    test_loss /= len(test_loader.dataset)
    accuracy = 100. * correct / len(test_loader.dataset)
    print(f'\nTest set: Average loss: {test_loss:.4f}, Accuracy: {correct}/{len(test_loader.dataset)} '
          f'({accuracy:.2f}%) \n')
```

```
torch.manual_seed(1)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

transform = transforms.Compose([
    transforms.ToTensor()
])

train_dataset = datasets.MNIST('./data', train=True, download=True, transform=transform)
test_dataset = datasets.MNIST('./data', train=False, transform=transform)

train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=64, shuffle=True)
test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=1000, shuffle=False)

model = SimpleCNN().to(device)

optimizer = optim.Adam(model.parameters(), lr=0.001)

epoches = 1
for epoch in range(1, epoches + 1):
    train(model, device, train_loader, optimizer, epoch)
    test(model, device, test_loader)
    torch.save(model.state_dict(), f'checkpoints/SimpleCNN_{epoch}.pth')
```

```
Train Epoch: 1 [0/60000 (0%)] Loss: 2.298472
Train Epoch: 1 [6400/60000 (11%)] Loss: 0.320231
Train Epoch: 1 [12800/60000 (21%)] Loss: 0.117048
Train Epoch: 1 [19200/60000 (32%)] Loss: 0.105377
Train Epoch: 1 [25600/60000 (43%)] Loss: 0.052762
Train Epoch: 1 [32000/60000 (53%)] Loss: 0.078863
Train Epoch: 1 [38400/60000 (64%)] Loss: 0.061150
Train Epoch: 1 [44800/60000 (75%)] Loss: 0.062919
Train Epoch: 1 [51200/60000 (85%)] Loss: 0.034232
Train Epoch: 1 [57600/60000 (96%)] Loss: 0.014758

Test set: Average loss: 0.0473, Accuracy: 9853/10000 (98.53%)
```

```
# 4. What's the predicted label of Lenna? What's the probability of that label (using softmax), is it high (e.g., >

model.load_state_dict(torch.load(f'/content/checkpoints/SimpleCNN_1.pth', weights_only=True))

for param in model.parameters():
    param.requires_grad = False

model.eval() # set model to evaluation mode

with torch.no_grad(): # no gradients needed
    output = model(lenna) # forward pass

    prob = F.softmax(output, dim=1) # convert to probabilities
```

```

        predicted_label = torch.argmax(prob, dim=1).item()
        confidence = torch.max(prob).item()

print("Predicted label:", predicted_label)
print("Confidence:", confidence)

```

```

Predicted label: 2
Confidence: 0.2638316750526428

```

```

# 5. Now modify the Lenna image to Lenna2 so as to maximize the probability of it being classified as 8.
# Run for say 1000 epochs
target = 8
lenna2 = torch.nn.Parameter(lenna.clone())
target = torch.tensor([target])
optimizer = optim.Adam([{"params":model.parameters()}, {"params": [lenna2]}], lr=0.01)

for epoch in range(1000):
    optimizer.zero_grad()
    lenna2.grad = None # this step is needed, it seems that previois step doesn't initialize the grad of data
    output = model(lenna2)
    # complete the rest
    loss = F.cross_entropy(output, target)
    loss.backward()
    optimizer.step()
    if epoch%10 == 0:
        print(f'Train Epoch: {epoch}, Loss: {loss.item():.6f}')

```

```

Train Epoch: 0, Loss: 2.146715
Train Epoch: 10, Loss: 0.119556
Train Epoch: 20, Loss: 0.015524
Train Epoch: 30, Loss: 0.005247
Train Epoch: 40, Loss: 0.003107
Train Epoch: 50, Loss: 0.002368
Train Epoch: 60, Loss: 0.001993
Train Epoch: 70, Loss: 0.001752
Train Epoch: 80, Loss: 0.001573
Train Epoch: 90, Loss: 0.001430
Train Epoch: 100, Loss: 0.001309
Train Epoch: 110, Loss: 0.001204
Train Epoch: 120, Loss: 0.001113
Train Epoch: 130, Loss: 0.001032
Train Epoch: 140, Loss: 0.000960
Train Epoch: 150, Loss: 0.000895
Train Epoch: 160, Loss: 0.000836
Train Epoch: 170, Loss: 0.000782
Train Epoch: 180, Loss: 0.000733
Train Epoch: 190, Loss: 0.000688
Train Epoch: 200, Loss: 0.000648
Train Epoch: 210, Loss: 0.000611
Train Epoch: 220, Loss: 0.000577
Train Epoch: 230, Loss: 0.000547
Train Epoch: 240, Loss: 0.000519
Train Epoch: 250, Loss: 0.000493
Train Epoch: 260, Loss: 0.000470
Train Epoch: 270, Loss: 0.000448
Train Epoch: 280, Loss: 0.000427
Train Epoch: 290, Loss: 0.000409
Train Epoch: 300, Loss: 0.000391
Train Epoch: 310, Loss: 0.000375
Train Epoch: 320, Loss: 0.000360
Train Epoch: 330, Loss: 0.000346
Train Epoch: 340, Loss: 0.000332
Train Epoch: 350, Loss: 0.000319
Train Epoch: 360, Loss: 0.000308
Train Epoch: 370, Loss: 0.000296
Train Epoch: 380, Loss: 0.000285
Train Epoch: 390, Loss: 0.000275
Train Epoch: 400, Loss: 0.000266
Train Epoch: 410, Loss: 0.000257
Train Epoch: 420, Loss: 0.000248
Train Epoch: 430, Loss: 0.000240
Train Epoch: 440, Loss: 0.000232
Train Epoch: 450, Loss: 0.000224
Train Epoch: 460, Loss: 0.000217
Train Epoch: 470, Loss: 0.000211
Train Epoch: 480, Loss: 0.000204
Train Epoch: 490, Loss: 0.000198
Train Epoch: 500, Loss: 0.000192
Train Epoch: 510, Loss: 0.000187
Train Epoch: 520, Loss: 0.000182
Train Epoch: 530, Loss: 0.000176
Train Epoch: 540, Loss: 0.000172
Train Epoch: 550, Loss: 0.000167
Train Epoch: 560, Loss: 0.000163

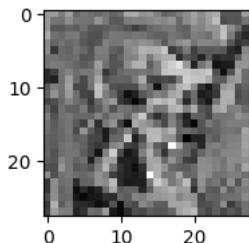
```

Train Epoch: 570 Loss: 0.000158

```
# show the lenna2 image
with torch.no_grad():
    output = model(lenna2)
    prob = F.softmax(output, dim=1)

print("Predicted label:", torch.argmax(prob).item())
print("Confidence:", torch.max(prob).item())
plt.figure(figsize=(2, 2))
# Use .detach() to allow conversion to numpy for plotting
plt.imshow(lenna2[0][0].detach().cpu().numpy(), cmap='gray')
plt.show()
```

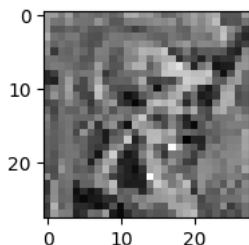
Predicted label: 8
Confidence: 0.9999349117279053



```
# 6. What's the predicted label of Lenna2? What's the probability of that label (using softmax), is it high?
with torch.no_grad():
    output = model(lenna2)
    prob = F.softmax(output, dim=1)

print("Predicted label:", torch.argmax(prob).item())
print("Confidence:", torch.max(prob).item())
plt.figure(figsize=(2, 2))
# Use .detach() to allow conversion to numpy for plotting
plt.imshow(lenna2[0][0].detach().cpu().numpy(), cmap='gray')
plt.show()
```

Predicted label: 8
Confidence: 0.9999349117279053



```
# 7. What's the difference between the original image and the new image in terms of mean square error (MSE)?
nn.MSELoss()(lenna2, lenna)
```

tensor(0.0265, grad_fn=<MseLossBackward0>)

```
# 8. Now create Lenna3, which not only maximizes the probability of it being classified as 8, but also minimize the c
lambda_reg = 0.1 # controls similarity strength
target = 8
lenna3 = torch.nn.Parameter(lenna.clone())
target = torch.tensor([target])
optimizer = optim.Adam([{"params":model.parameters()}, {"params": [lenna3]}], lr=0.01)

for epoch in range(5000):
    optimizer.zero_grad()
    lenna3.grad = None # this step is needed, it seems that previous step doesn't initialize the grad of data
    output = model(lenna3)
    class_loss = F.cross_entropy(output, target)
    mse_loss = F.mse_loss(lenna3, lenna)
    loss = class_loss + lambda_reg * mse_loss
    # loss = ??(output, target) + ??(lenna3, lenna)
    loss.backward()
    optimizer.step()
    if epoch%500 == 0:
        print(f'Train Epoch: {epoch}, Loss: {loss.item():.6f}')
output = model(lenna3)
prob = F.softmax(output, dim=1)
```

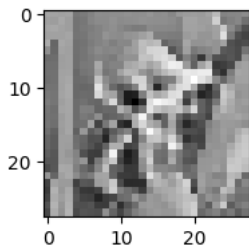
```
print("Predicted label:", torch.argmax(prob).item())
print("Confidence:", torch.max(prob).item())
```

```
Train Epoch: 0, Loss: 2.146715
Train Epoch: 500, Loss: 0.001866
Train Epoch: 1000, Loss: 0.001485
Train Epoch: 1500, Loss: 0.001314
Train Epoch: 2000, Loss: 0.001227
Train Epoch: 2500, Loss: 0.001175
Train Epoch: 3000, Loss: 0.001142
Train Epoch: 3500, Loss: 0.001129
Train Epoch: 4000, Loss: 0.001123
Train Epoch: 4500, Loss: 0.001121
Predicted label: 8
Confidence: 0.9998268485069275
```

```
# 9. What's the predicted label of Lenna3? What's the probability of that label (using softmax)? Is it high?
output = model(lenna3)
prob = F.softmax(output, dim=1)

print("Predicted label:", torch.argmax(prob).item())
print("Confidence:", torch.max(prob).item())
plt.figure(figsize=(2, 2))
# Use .detach() to allow conversion to numpy for plotting
plt.imshow(lenna3[0][0].detach().cpu().numpy(), cmap='gray')
plt.show()
```

```
Predicted label: 8
Confidence: 0.9998268485069275
```



```
# 10. What's the difference between the Lenna and Lenna3 in terms of mean square error (MSE)?
# How does it compare with MSE between lenna and lenna2
mse_lenna2 = F.mse_loss(lenna2, lenna.to(device)).item()
mse_lenna3 = F.mse_loss(lenna3, lenna.to(device)).item()
```

```
print("MSE (lenna vs lenna2):", mse_lenna2)
print("MSE (lenna vs lenna3):", mse_lenna3)
```

```
MSE (lenna vs lenna2): 0.026543933898210526
MSE (lenna vs lenna3): 0.00945592112839222
```

```
# 11. plot Lenna, Lenna2, Lenna3 side-by-side, and offer your observation of whether your attack is successful
# (able to for any label yet the image still looks similar to original)
import matplotlib.pyplot as plt
```

```
fig, axes = plt.subplots(1, 3, figsize=(12, 4))
```

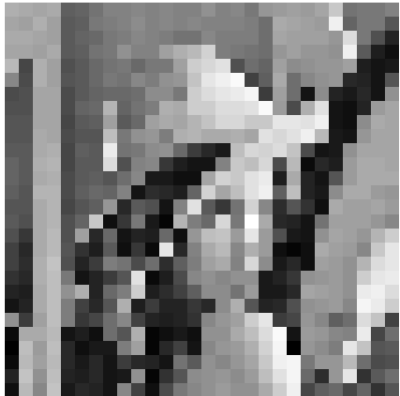
```
# Original
axes[0].imshow(lenna[0][0].detach().cpu().numpy(), cmap='gray')
axes[0].set_title("Original (Lenna)")
axes[0].axis('off')
```

```
# Lenna2 (no regularization)
axes[1].imshow(lenna2[0][0].detach().cpu().numpy(), cmap='gray')
axes[1].set_title("Lenna2 (No Reg)")
axes[1].axis('off')
```

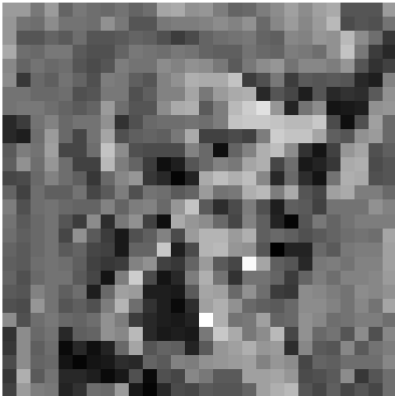
```
# Lenna3 (with regularization)
axes[2].imshow(lenna3[0][0].detach().cpu().numpy(), cmap='gray')
axes[2].set_title("Lenna3 (With Reg)")
axes[2].axis('off')
```

```
plt.tight_layout()
plt.show()
```

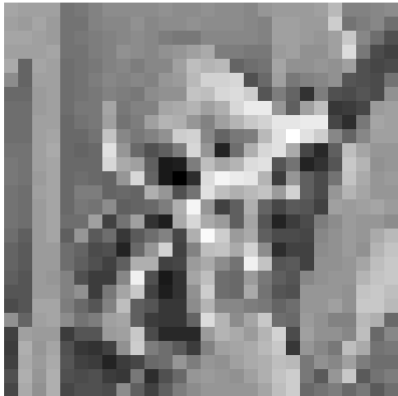
Original (Lenna)



Lenna2 (No Reg)



Lenna3 (With Reg)



开始借助 AI 编写或生成代码。

开始借助 AI 编写或生成代码。